you can minimize unnecessary database calls, you are less likely to suffer from performance issues.

Server-side caching is primarily used to improve performance in applications. This type of caching involves the storage of data or content on the server, particularly data or content that incurs a processing overhead, so it can be reused without incurring the cost of generation every time. In-memory storage is most likely to be used in applications hosted on a single server. Applications deployed to the cloud or on web farms are advised to use a distributed cache system—one that is separate from the application, so each instance of the application can access the same cached data. In this section, we will cover in-memory storage.

TIP The official documentation provides guidance on distributed cache strategies, including the use of two popular solutions, Redis and NCache, along with one designed to work with SQL Server (http://mng.bz/z5g1).

The best candidate for caching is data that doesn't change very often. Lookup data, tag clouds, and menu-driven navigation are all classic examples that can benefit from caching. Remember your custom route constraint from chapter 4, where you validated the route data against an array of cities? We discussed doing this against the database, rather than a hardcoded list. In this section, you will cache the city data from the database, refreshing it when it changes, so the custom route constraint is always up to date without becoming a burden on the database.

Razor Pages offers two main ways you can manage the caching of pieces of content. One is using a cache provider's API, such as that offered by the `IMemoryCache` service. You will leverage this soon enough for the use case just discussed. Before that, you will take a quick look at the other option: the cache tag helper, which is designed to be used within a Razor page itself.